



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 7

Recap - Planning and scheduling



Scheduling in Waterfall

- **Upfront Planning:** Scheduling is detailed and occurs at the project's start, with all tasks, milestones, and dependencies defined early using tools like WBS, Gantt charts, and critical path analysis with AoN diagrams..
- **Fixed Timeline:** Once set, the schedule is rigid, with phases (e.g., requirements, design, implementation) completed sequentially.
- **Dependency-Driven:** Tasks follow a strict sequence, with the critical path determining the minimum project duration.
- **Progress Monitoring:** Measured against the initial plan, with little flexibility for changes.
- **Challenges:** Suited for stable requirements but risks delays if unforeseen issues arise.

Recap - Planning and scheduling



Scheduling in Scrum

- **Iterative Planning:** Scheduling is incremental, with plans evolving through 2-4 week sprints.
- **Flexible Timeline:** Release dates are estimates, adjusted based on sprint velocity and feedback.
- **Sprint-Based:** Each sprint has a goal with schedules refined during sprint planning.
- **Progress Monitoring:** Tracked with burndown charts and sprint reviews allowing real-time adjustments.
- **Challenges:** Requires active team collaboration and may lack long-term visibility, necessitating a release plan.

Quality Planning

Quality Planning



- Quality management
- Quality planning
 - Identification relevant quality attributes
 - Defining associated measures
 - Setting up standards for the product and processes

Quality



- Quality, simplistically, means that a product should meet its specification (i.e., meets functional needs and adheres to standards)
- The software product should deliver the required functionality (*functional requirements*) with the required **quality attributes** (*non-functional requirements*)

Quality management



- **Quality planning**
 - **Setting quality standards and developing a plan to achieve them**
- **Quality assurance**
 - Ensuring that the development process is compliant with the quality plan
- **Quality control**
 - Monitoring and evaluating the product to ensure that it meets the quality requirements

Further discussion assurance and control is planned during the second half of this course

Product and process standards –



Some examples

Product standards

Design review form

Requirements document structure

Programming style

Project plan format

Change request form

Process standards

Design review conduct

Submission of new code for system building

Project plan approval process

Change control process

Test recording process

Quality planning



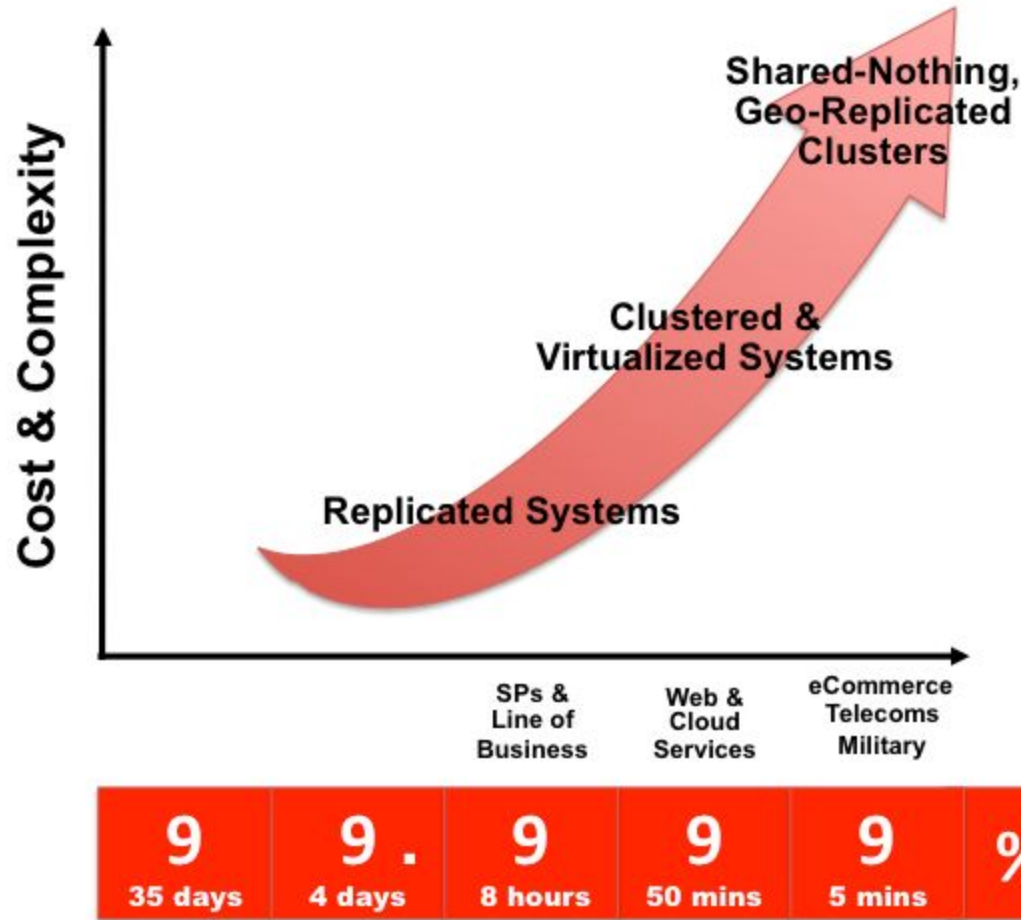
- Identifies the most significant quality attributes appropriate for the product
- Defines the assessment process in detail for each quality attribute (including goals or target values)
- Indicates which standards should be applied and defines new standards as necessary

Quality attribute example



- Reliability – probability of system working satisfactorily within a specific period of time
- Possible measures:
 - Availability: % of a time that a system is usable
 - Mean time between failures: total service time/number of failures
 - Failure on demand: probability that system will not available when needed
 - Support activity: number of fault reports generated and processed

Trade-offs: Cost & Complexity vs. Availability



https://docs.oracle.com/cd/E17952_01/mysql-5.5-en/ha-overview.html

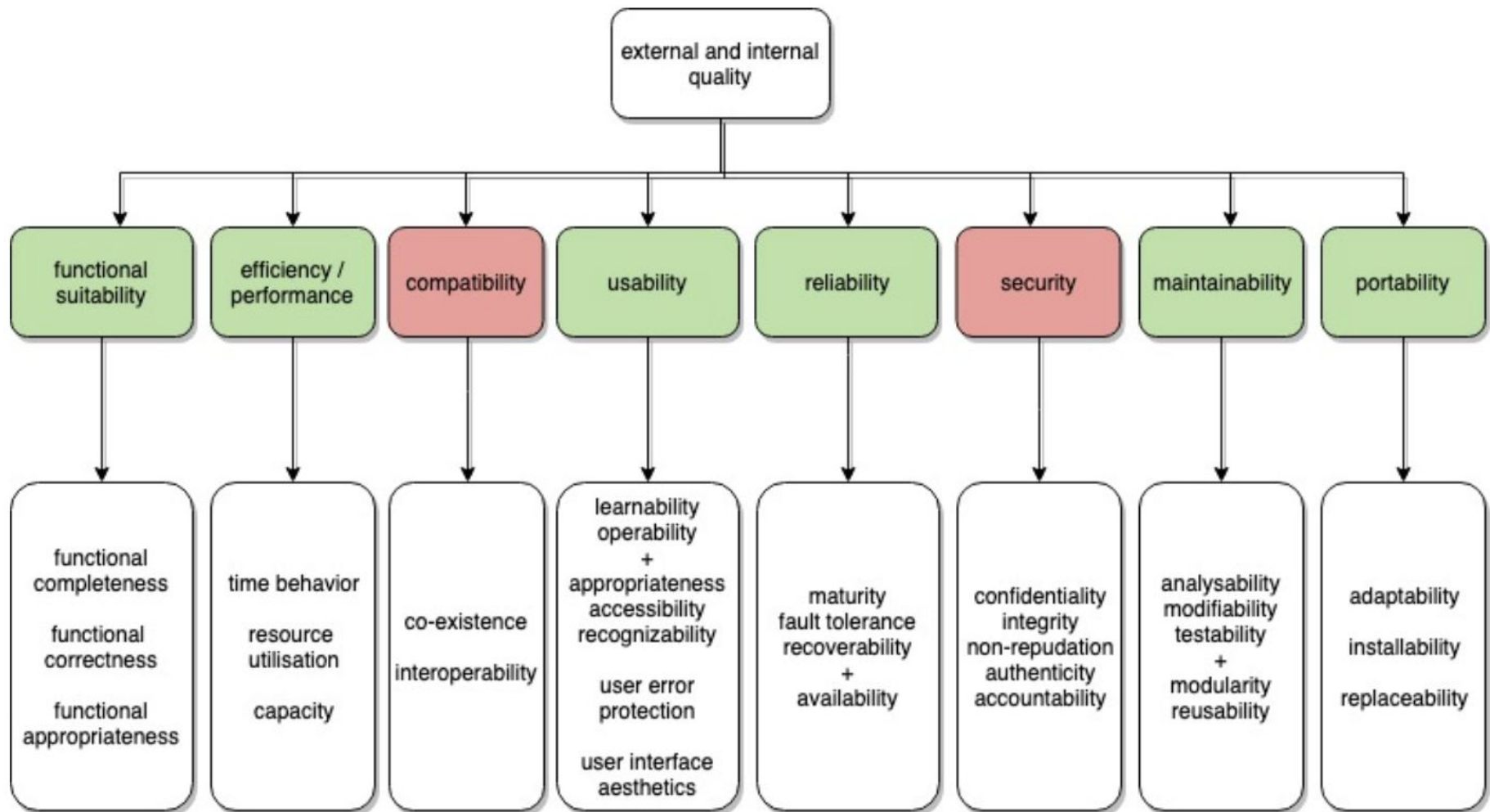
Software Quality Attributes

- Safety
- Security
- Reliability
- Resilience
- Robustness
- Understandability
- Testability
- Adaptability
- Modularity
- Complexity
- Portability
- Usability
- Reusability
- Efficiency
- Learnability

ISO/IEC 25010 standard



<https://www.monterail.com/blog/software-ga-standards-iso-25010>



ISO/IEC 25010 categorization of software quality requirements Source: ISO20500.com

ISO/IEC 25010 - Product Quality model



Focuses on the static properties of a product, including its eight main quality characteristics (listed below) and their sub-characteristics:

- **Functional suitability:** Functions performed by the system fulfill user needs.
- **Performance efficiency:** How well the system performs its functions in terms of time and resources.
- **Compatibility:** The ability of the system to exchange information with other systems or operate in a shared environment.
- **Usability:** How easy the system is to use and understand by users.
- **Reliability:** The ability of the system to maintain its level of performance under specific conditions.
- **Security:** How well the system protects data and resources from unauthorized access and breaches.
- **Maintainability:** The ease with which the system can be modified.
- **Portability:** The ease with which the system can be transferred from one environment to another.

Quality attributes and some related measures/metrics



Quality Attribute	Description	Measure/metric Example
Reliability	System uptime and error handling	Mean Time Between Failures (MTBF)
Usability	Ease of use	User satisfaction surveys
Performance	Speed and efficiency	Response time metrics

Activity - W07#1

Suggest measures/metrics corresponding to the eight quality attributes listed for Mux-Core system

Some common practices for selecting attributes and metrics



- **Functional Suitability:** 1-2 metrics (e.g., Percentage of requirements fulfilled, Functional completeness).
- **Performance Efficiency:** 2-3 metrics (e.g., Response time, Throughput, Resource utilization).
- **Compatibility:** 1-2 metrics (e.g., Integration success rate, Error rate in data exchange).
- **Usability:** 2-3 metrics (e.g., Task completion time, User satisfaction score, Error frequency).
- **Reliability:** 2-3 metrics (e.g., Mean Time Between Failures (MTBF), Availability percentage, Failure rate).
- **Security:** 2-3 metrics (e.g., Vulnerability detection rate, Data breach incidents, Compliance rate).
- **Maintainability:** 1-2 metrics (e.g., Change request resolution time, Code complexity).
- **Portability:** 1-2 metrics (e.g., Adaptation effort, Platform compatibility rate).

Typical quality planning activities in waterfall



Waterfall's linear approach allows for comprehensive upfront quality planning, ensuring all quality aspects are defined early in the project. Key activities include:

- **Define Quality Attributes and Metrics:** Establish specific quality standards (e.g., ISO 25010 attributes) and metrics (e.g., defect rate < 5%, system uptime > 99.9%) for the system.
- **Develop Quality Management Plan:** Create a detailed plan outlining quality objectives, roles, and processes (e.g., reviews, audits) for each phase—requirements, design, implementation, testing, and deployment.
- **Conduct Requirements Validation:** Perform thorough reviews and walkthroughs of requirements with stakeholders to ensure quality is built into the system design, addressing issues like integration with other systems.
- **Plan Testing Phases:** Schedule distinct testing phases (unit, integration, system, acceptance) with predefined test cases, ensuring the system meets performance and security needs.
- **Risk-Based Quality Prioritization:** Identify potential quality risks (e.g., legacy system failures) and prioritize quality efforts accordingly, documenting mitigation strategies in the plan.

Typical quality planning activities in Scrum



Scrum's iterative nature integrates quality planning into each sprint, making it adaptable for the project's evolving requirements. Key activities include:

- **Define Sprint Quality Goals:** Set quality objectives for each sprint, aligning with user stories and the Definition of Done (DoD).
- **Incorporate Quality in Backlog Refinement:** Prioritize quality-related tasks (e.g., automated testing scripts) in the product backlog, ensuring continuous attention to quality during sprints.
- **Implement Continuous Testing:** Conduct unit tests, integration tests, and acceptance tests within each sprint using automated tools, validating features like integrating with external systems.
- **Conduct Sprint Reviews for Quality Feedback:** Engage stakeholders in sprint reviews to assess quality, gathering feedback on usability and performance to refine future sprints.
- **Use Retrospectives for Quality Improvement:** Analyze quality issues (e.g., bugs in systems integration) during retrospectives, planning improvements for the next sprint to enhance the system's reliability.

Risk Management

Risk management



- Software projects are inherently complex and uncertain due to changing requirements, evolving technology and dependence on multiple stakeholders
- Effective risk management helps to anticipate and address potential issues before they become major problems
- Ensures project success by minimizing the impact of negative risks and maximizing the potential of positive risks

Risk Management



- Risk identification – what are the risks to a project?
- Risk analysis – which ones are really serious?
- Risk planning – what shall we do?
- Risk monitoring – has the planning worked?

Risk Identification



- **Use of checklists** – usually based on the experience of past projects
- **Brainstorming** – getting knowledgeable stakeholders together to pool concerns
- **Causal mapping** – identifying possible chains of cause and effect

Activity W07#2: Identify possible risks for the Mux-Core project



Visit <http://www.projectengineer.net/project-risk-checklist/> and identify five or six possible risks relevant for the Mux-Core project

Post your suggestions with justification with the probability of its occurrence high/medium/low and severity or impact (e.g., high > 0.8 , medium > 0.4 , low ≤ 0.4)

E.g., scope creep (medium) – many of the requirements related to administration and counter ticket sales are fairly well understood; but, the mobile/web for customers and management support need more clarity

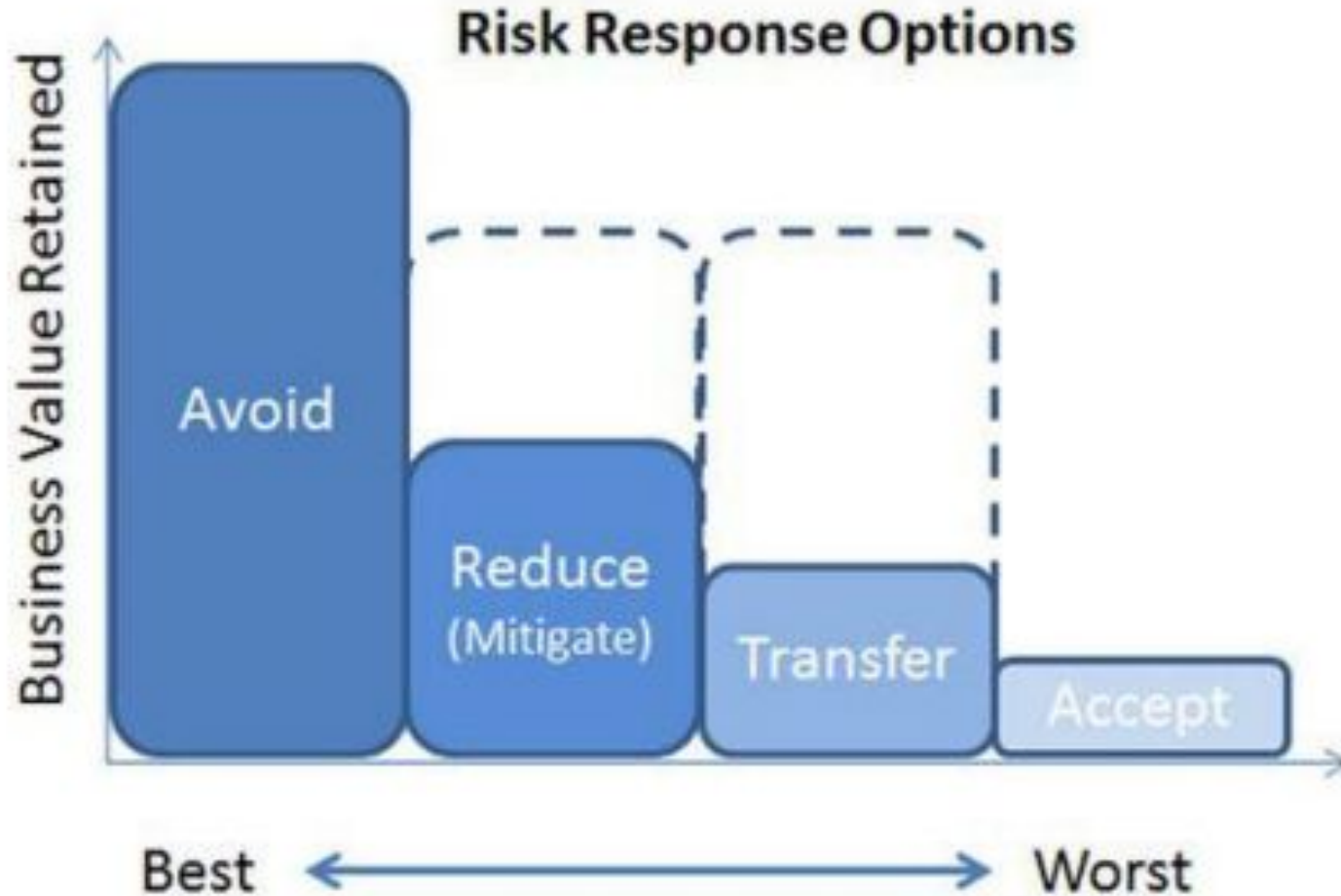
(Try different risk categories; Do not repeat the entries posted by others; instead improve those entries)

Risk mitigation - examples



Risk	Probability (High/Med/ Low)	Impact (High/Med/ Low)	Mitigation Strategy	Monitoring Metric
Changing requirements	High	High	Regular stakeholder reviews	Number of change requests per sprint
Technology obsolescence	Med	High	Enhance vendor relationship	Tech stack audit frequency
Stakeholder delays	Med	Med	Contingency buffers	Milestone completion rates

Response options



Risk Probability and Severity



		Severity		
		Low	Medium	High
Probability	High	Medium risk	High risk	High risk
	Medium	Low risk	Medium risk	High risk
	Low	Low risk	Low risk	Medium risk
		Low risk	Medium risk	High risk

Activity W07#3: Suggest how the following risks can be managed



- Scope creep
- Quality of product does not meet standards
- Project is running behind schedule

Suggest **reduce (mitigate)** strategies for managing above risks

Commonly used risk items in software development projects



Small Projects (e.g., <6 months, <5 team members):

10-12 risks, focusing on key areas like resource availability and technical feasibility.

Medium Projects (e.g., 6-12 months, 5-15 team members): 15-20 risks, addressing scope creep, integration challenges, and schedule delays.

Large Projects (e.g., >12 months, >15 team members): 20+ risks, including strategic, operational, and external factors.

Typical risk management activities in Waterfall



Waterfall's sequential nature allows for a structured, upfront approach to risk management. Key activities include:

- **Risk Identification:** Conduct a comprehensive risk assessment during the planning phase, identifying potential risks such as legacy system integration failures, stakeholder resistance, or project delays.
- **Risk Analysis:** Perform a detailed analysis using a probability-impact matrix to prioritize risks (e.g., infrastructure downtime, staff availability issues), documenting findings in a risk register.
- **Risk Planning:** Develop a risk mitigation plan with specific strategies, such as allocating contingency buffers for delays or conducting pilot tests on legacy systems before full integration.
- **Risk Monitoring and Control:** Establish checkpoints at the end of each phase (e.g., design, implementation) to review risks, updating the risk register and adjusting plans as needed during the system development.
- **Documentation and Reporting:** Maintain detailed documentation of all risks, mitigation actions, and outcomes, ensuring traceability and providing a reference for future projects.

Typical risk management activities in Scrum



Scrum's iterative approach integrates risk management into each sprint, allowing for continuous identification and adaptation. Key activities include:

- **Risk Identification in Sprint Planning:** Identify risks specific to each sprint (e.g., scope creep or technology obsolescence) during planning sessions.
- **Risk-Adjusted Backlog Prioritization:** Adjust the product backlog to address high-priority risks early, such as testing integration with external systems in the initial sprints to mitigate connectivity issues.
- **Mitigation During Sprints:** Implement risk mitigation strategies incrementally, such as prototyping uncertain components within a sprint and reviewing outcomes in daily stand-ups.
- **Risk Review in Sprint Retrospectives:** Assess the effectiveness of risk management strategies post-sprint, identifying new risks (e.g., staff burnout) and planning improvements for the next sprint.
- **Visual Risk Tracking:** Use tools like risk burndown charts or Kanban boards to monitor risks visually, updated in real-time fostering team awareness.

Some risk management tools and platforms



- Microsoft Project: Offers comprehensive risk tracking and reporting for large-scale projects.
- Primavera Risk Analysis: Specializes in quantitative risk analysis for complex projects with many variables.
- Celoxis: A platform for detailed risk analysis, monitoring, and integration with other project functions.
- Asana and Trello: Used for tracking project progress and mitigating risks.
- ProofHub: A centralized platform to identify, assign, and track risks, converting them into actionable tasks.